# *rpcgen*: The Simple Way to Develop Distributed Applications

In distributed computing, the data and the services processing the data are placed in different computers connected by a network. This makes developing a distributed application a daunting task. This article presents a simple method for developing a distributed application using *rpcgen*.

Many scientific problems are solved using distributed systems, as shown in Figure 1. Basically, a distributed system is a collection of autonomous computers connected by a network that appears to the users of the system as a single computer. The message passing mechanism plays a major role in the success of distributed computing. For a distributed application, developers have to worry about details such as sockets, network byte order, host byte order, etc. In this situation, Remote Procedure Call (RPC) presents a simple mechanism for distributed computing without the need to include socket code in the distributed application.

## *rpcgen*

Open Network Computing (ONC) Remote Procedure Call (RPC) is a widely deployed remote procedure call system. ONC was originally developed by Sun Microsystems in the 1980s as part of the company's Network File System project, and is sometimes referred to as Sun RPC. *rpcgen* is an interface generator precompiler for Sun Microsystems ONC RPC. Now, *rpcgen* is supported by many systems. With the availability of the *rpcgen* application, developers can concentrate on developing the core features of their application, instead of spending most of their time on developing and debugging their network interface code.

## Application development using RPC

Distributed computing uses the divide and conquer principle to solve computationally-intensive problems. The problem is divided into many tasks, each of which is computed by one or more computers in the distributed systems. For the sake of simplicity, let us start developing a distributed application with two computing services called *max* and *min* running on two different computers, as shown in Figure 2. The service *max* finds the maximum number in the given list of numbers and the service *min* finds the minimum number in the given list of numbers.

Distributed application development, using *rpcgen*, starts with designing the Interface Definition Language file. This file contains the details about the services that run on different autonomous computers in the distributed system. The file has to be saved with a *.x* extension like *dist_app.x*, as in this case.

```
struct ipair   {   // Data that will be processed by the
services
            int data[10];
        int count;
  };
program MINMAX{       // MINMAX is the name of interface
    version VER1    {       //VER1 is the version
          int max(ipair)=1;
          int min(ipair)=2;
    }=1;
}=0x3a3afeeb;       // 32 bit program number
```

The next step is compiling *dist_app.x* with *rpcgen*.

$*rpcgen –C dist_app.x* as shown in Figure 3 creates four new files—*dist_app_clnt.c*, *dist_app.h*, *dist_app_svc.c* and *dist_app_xdr.c*. The option *–C* is used to generate ANSI C code. The files *dist_app_clnt.c* and *dist_app_svc.c* are called the client stub and server stub, respectively. These are responsible for maintaining networking connections. The file *dist_app.h* is the header file which will be included in the distributed application. The file *dist_app_xdr.c* is called the *eXternal data representation* file. It will help in data conversions.

The next step is to create the distributed application's sample client and sample server programs using the options *–Sc* and *–Ss* with *rpcgen*, as shown in Figure 4.

We then slightly modify the sample files *dist_app_client.c*, *dist_app_server1.c* and *dist_app_server2.c* according to our